Formal Mathematical Specifications of Cryptographic Hardware Cores

Hardware Architecture Documentation

November 4, 2026

Abstract

This document provides the formal mathematical specifications corresponding to the Verilog hardware implementations of three cryptographic primitives: the SHA-256 compression core, the SHA-512 compression core, and the Keccak-f[1600] sponge core (SHA-3). Each core is modeled mathematically over its respective algebraic ring, detailing bitwise operators, message schedule expansions, round-based state transitions, and finalization steps.

1 SHA-256 Compression Core

All arithmetic operations for the SHA-256 core are defined over the ring $\mathbb{Z}_{2^{32}}$, representing unsigned 32-bit addition modulo 2^{32}.

1.1 Bitwise Functions and Operators

Let $x, y, z \in \{0,1\}^{32}$.

- **Right Rotation:** $\mathrm{ROTR}^n(x) = (x \gg n) \vee (x \ll (32-n))$
- **Right Shift:** $\mathrm{SHR}^n(x) = x \gg n$
- **Message Schedule Auxiliary Functions:**

$$\sigma_0(x) = \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x)$$
$$\sigma_1(x) = \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x)$$

- **Compression Auxiliary Functions:**

$$\Sigma_0(x) = \mathrm{ROTR}^2(x) \oplus \mathrm{ROTR}^{13}(x) \oplus \mathrm{ROTR}^{22}(x)$$
$$\Sigma_1(x) = \mathrm{ROTR}^6(x) \oplus \mathrm{ROTR}^{11}(x) \oplus \mathrm{ROTR}^{25}(x)$$
$$\mathrm{Ch}(x,y,z) = (x \wedge y) \oplus (\neg x \wedge z)$$
$$\mathrm{Maj}(x,y,z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$$

1.2 Message Schedule Expansion

Given a padded 512-bit block $M = M^{(0)} \parallel M^{(1)} \parallel \cdots \parallel M^{(15)}$ where each $M^{(i)} \in \{0,1\}^{32}$, for round index $t \in \{0, 1, \ldots, 63\}$, the expanded word W_t is defined as:

$$W_t = \begin{cases} M^{(t)}, & 0 \le t \le 15 \\ (\sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}) \pmod{2^{32}}, & 16 \le t \le 63 \end{cases} \tag{1}$$

1.3 Round Function State Update

Let the working state at round t be $V_t = (a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t) \in (\{0,1\}^{32})^8$.
Initialization ($t = 0$): $V_0 = H^{(i-1)}$, selected via the chaining configuration ('cont' flag or initial hash vector H_{256}).
State Transition ($t \to t+1$ for $t = 0 \dots 63$):

$$T_1^{(t)} = (h_t + \Sigma_1(e_t) + \mathsf{Ch}(e_t, f_t, g_t) + K_t + W_t) \pmod{2^{32}}$$
$$T_2^{(t)} = (\Sigma_0(a_t) + \mathsf{Maj}(a_t, b_t, c_t)) \pmod{2^{32}}$$

The register updates follow:

$$a_{t+1} = T_1^{(t)} + T_2^{(t)} \pmod{2^{32}}, \quad b_{t+1} = a_t, \quad c_{t+1} = b_t, \quad d_{t+1} = c_t$$
$$e_{t+1} = d_t + T_1^{(t)} \pmod{2^{32}}, \quad f_{t+1} = e_t, \quad g_{t+1} = f_t, \quad h_{t+1} = g_t$$

1.4 Finalization & Digest Computation

After 64 rounds, the output digest $H^{(i)}$ is computed by modular addition of the final state variables with the initial chaining values $H^{(i-1)}$:

$$H_j^{(i)} = H_j^{(i-1)} + \text{state_variable}_{64} \pmod{2^{32}}, \quad \text{for } j \in \{0, \dots, 7\} \tag{2}$$

2 SHA-512 Compression Core

All arithmetic operations for the SHA-512 core are defined over the ring $\mathbb{Z}_{2^{64}}$, representing unsigned 64-bit addition modulo 2^{64}.

2.1 Bitwise Functions and Operators

Let $x, y, z \in \{0,1\}^{64}$.

- **Right Rotation:** $\mathsf{ROTR}^n(x) = (x \gg n) \vee (x \ll (64 - n))$
- **Message Schedule Auxiliary Functions:**

$$\sigma_0(x) = \mathsf{ROTR}^1(x) \oplus \mathsf{ROTR}^8(x) \oplus \mathsf{SHR}^7(x)$$
$$\sigma_1(x) = \mathsf{ROTR}^{19}(x) \oplus \mathsf{ROTR}^{61}(x) \oplus \mathsf{SHR}^6(x)$$

- **Compression Auxiliary Functions:**

$$\Sigma_0(x) = \mathsf{ROTR}^{28}(x) \oplus \mathsf{ROTR}^{34}(x) \oplus \mathsf{ROTR}^{39}(x)$$
$$\Sigma_1(x) = \mathsf{ROTR}^{14}(x) \oplus \mathsf{ROTR}^{18}(x) \oplus \mathsf{ROTR}^{41}(x)$$
$$\mathsf{Ch}(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$$
$$\mathsf{Maj}(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$$

2.2 Message Schedule and Round Updates

Given a 1024-bit block M, words W_t are expanded for $t \in \{0, \dots, 79\}$ identically to SHA-256 but modulo 2^{64}. The round update equations mirror SHA-512 specifications operating over 64-bit words with 80 total rounds.

3 Keccak-f[1600] Sponge Core (SHA-3)

The internal state is represented as a 3D array $A[x, y][z]$ or a 5×5 grid of 64-bit lanes ($S \in \{0, 1\}^{1600}$).

3.1 State Absorption Phase

When a block is absorbed, the state is conditionally initialized and XOR-ed with the rate-sized input block M:

$$S^{(0)} = \begin{cases} 0^{1600}, & \text{if init} = 1 \\ S_{\text{prev}}, & \text{if init} = 0 \end{cases} \tag{3}$$

For each byte index i up to the rate size bs:

$$S_{\text{byte},i} \leftarrow S^{(0)}_{\text{byte},i} \oplus M_{\text{byte},i} \tag{4}$$

3.2 Keccak-f[1600] Round Permutation

Each round consists of five transformations executed sequentially over 24 rounds ($r \in \{0, \ldots, 23\}$):

1. **Theta (θ):** Parity calculation and diffusion across columns:

$$C[x] = \bigoplus_{y=0}^{4} A[x, y], \quad D[x] = C[(x-1) \bmod 5] \oplus \text{ROTL}^1\left(C[(x+1) \bmod 5]\right)$$
$$A[x, y] \leftarrow A[x, y] \oplus D[x]$$

2. **Rho (ρ) & Pi (π):** Lane offset relocation and rotation.

3. **Chi (χ):** Non-linear row-wise combination:

$$A[x, y] = B[x, y] \oplus (\neg B[(x+1) \bmod 5, y] \wedge B[(x+2) \bmod 5, y])$$

4. **Iota (ι):** Round constant XOR into lane $A[0, 0]$ using $\text{RC}(r)$.

3.3 Digest Extraction

Upon completion of 24 rounds, the 256-bit digest is extracted from the lower 256 bits of the final state vector:

$$\text{Digest} = S^{(24)}[255 : 0] \tag{5}$$